

REGRESSION BY SUCCESSIVE ORTHOGONALISATION

RECAP

Linear regression: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \epsilon$

In matrix form $Y = XB + \epsilon$

We estimate $\beta = [\beta_0 \ \beta_1 \ \dots \ \beta_p]^T$ by $\hat{\beta} = [\hat{\beta}_0 \ \hat{\beta}_1 \ \dots \ \hat{\beta}_p]$

minimising $RSS(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (+ \lambda \sum_{j=1}^p \beta_j^2)$

Setting $\frac{\partial RSS}{\partial \beta} = 0 \Rightarrow \hat{\beta} = (X^T X)^{-1} X^T Y$

$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T Y$ (Ridge)

Regression by successive orthogonalisation

Model: $y_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} + \epsilon_i$

Goal: compute $\hat{\beta} = [\beta_0 \ \beta_1 \ \dots \ \beta_p]^T$

Step 1 | For each predictor

compute: $\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$

and we center the data. For every observation:

$\tilde{x}_{ij} = x_{ij} - \bar{x}_j$ { Similar to
 $\tilde{y}_i = y_i - \bar{y}$ { normalising

No intercept!
Can you see why?

The regression task becomes $\tilde{y}_i = \sum_{j=1}^p \beta_j \tilde{x}_{ij} + \epsilon$

Step 2 | Core idea

Two predictors are orthogonal if $x_j^T x_k = 0$ ($k \neq j$).

All predictors are orthogonal if $x_j^T x_k = 0 \ \forall k, j$ with $k \neq j$

If that is the case, then $\beta_j = \frac{x_j^T y}{x_j^T x_j}$

Idea: build new orthogonal vectors $z_1, \dots, z_p$

Step 3 | Core algorithm (Gram-Schmidt)

1) Set $z_1 = \tilde{x}_1$

2) Compute z_2 :

Remove from $\tilde{x}_2$ the part explained by z_1 .

$$a_{12} = \frac{z_1^T \tilde{x}_2}{z_1^T z_1}, \quad z_2 = \tilde{x}_2 - a_{12} z_1$$

You can check $z_1^T z_2 = 0 \rightarrow$ Orthogonal

3) For z_3 (and z_j)

$$z_3 = \tilde{x}_3 - a_{13} z_1 - a_{23} z_2, \quad a_{kj} = \frac{z_k^T \tilde{x}_j}{z_k^T z_k}$$

Generally,

$$z_j = \tilde{x}_j - a_{1j} z_1 - a_{2j} z_2 - \dots - a_{(j-1)j} z_{(j-1)}$$

Pseudocode:

For j in $1 \dots p$:

$z_j = \tilde{x}_j$

for k in $1 \dots j-1$:

$$a_{kj} = \text{dot}(z_k, \tilde{x}_j) / \text{dot}(z_k, z_k)$$

$$z_j = z_j - a_{kj} z_k$$

Step 4 | Now:

$$\tilde{y} = \sum_{j=1}^p \hat{\gamma}_j z_j \quad \hat{\gamma}_j = \frac{z_j^T \tilde{y}}{z_j^T z_j} \quad (\hat{\beta} \neq \hat{\gamma})$$

We can get predictions, but how do we obtain the original $\hat{\beta}$?

Step 5 | In matrix form, we have:

$$\tilde{X} = ZR \quad R = \begin{bmatrix} 1 & a_{12} & a_{13} & \dots \\ 0 & 1 & a_{23} & \dots \\ 0 & 0 & 1 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

$$\text{and } y = R\beta \Rightarrow \beta = R^{-1} \gamma$$

Back substitution: $\rightarrow$ Since R is upper triangular

$$\hat{\beta}_p = \gamma_p \quad (\text{Start with last predictor})$$

$$\hat{\beta}_j = \gamma_j - \sum_{k=j+1}^p R_{jk} \hat{\beta}_k$$

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^p \hat{\beta}_j \bar{x}_j$$

Why bother?

- Numerically stable, faster, standard in numerical libraries.
- Statistical interpretation of OLS

Exercises

1) Implement functions to center a vector and center the columns of a matrix.

2) Implement Gram-Schmidt

Input: X
Output: Z, R { Verify $X \approx Z @ R$

3) Compute the $\hat{\gamma}$ coefficients

4) Obtain $\hat{\beta}$

5) Create a full OLS via orthogonalisation function and compare your results with $\hat{\beta} = (X^T X)^{-1} X^T Y$ (Vanilla OLS)